Day 97 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 7 reference

Watch: Measuring LLM Accuracy with BLEU and ROUGE score (DATAJARVIS) | Language Model Evaluation and Perplexity (Machine Learning TV)

`day97_lm_evaluation.py`, segment by segment, in the order `main()` uses things. The six exercise parts (Days 91-96) are copied verbatim and were explained on their own days; this guide recaps each in one segment, then walks the new code: BLEU, ROUGE and the evaluation. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: p_n = clipped n-gram precision · BP = brevity penalty · c, r = hypothesis and reference lengths in words · LCS = longest common subsequence · bits = mean $-\log_2 p$ per character

Segment 0 — Setup

Imports and settings

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day96_fine_tuning as d96
d90 = d96.d90
MODELS = {"Day 92 (general)": "day92_mini_gpt.npz", "Day 96 (fine-tuned)":
    "day96_finetuned_gpt.npz"}
N_PROMPTS, PROMPT_LEN, CONT_LEN = 24, 64, 150
SEEDS = [0, 1, 2]
KN_ORDER, KN_D = 7, 0.9
FT_STEPS, FT_CHECK, FT_LR = 200, 50, 3e-4
SPEAKER = re.compile(r"^[A-Z][A-Z' ]+:$")
```

- `import day96_fine_tuning as d96` — Day 96's script gives The Tempest's split; through it, Day 90's GPT code (d96.d90).
- `MODELS = {...}` — the two saved files: Day 92's general model and Day 96's fine-tuned one.
- `SPEAKER = re.compile(...)` — a module-level constant that Day 94's speaker_share needs, copied with it.

Segment 1 — The week's parts

The parts, verbatim: one banner per exercise script

CODE

```
# ---- from day96_exercise.py, verbatim ----
def leaves(D, prefix=""):
    for k, v in D.items():
        if isinstance(v, dict):
            yield from leaves(v, prefix + k + ".")
        else:
            yield prefix + k, v
def save_params(path, P):
    np.savez_compressed(path, **{k.replace(".", "__"): v for k, v in leaves(P)})
def load_params(path):
    P, f = {}, np.load(path)
    for k in f.files:
        parts = k.split("__"); node = P
        for part in parts[:-1]:
            node = node.setdefault(part, {})
        node[parts[-1]] = f[k]
    return P
```

- `# ---- from day96_exercise.py, verbatim ----` — the banner names the script each function was copied from, unchanged.
- `yield from leaves(v, prefix + k + ".")` — a generator that walks nested dictionaries and yields ('blocks.0.Wq', array) pairs.
- `node = node.setdefault(part, {})` — load_params rebuilds the nesting: 'blocks__0__Wq' becomes P['blocks']['0']['Wq'].

Segment 2 — BLEU and ROUGE (new)

Words and n-grams

CODE

```
def words(text):
    return re.findall(r"[a-z']+", text.lower())
def ngrams(ws, n):
    return Counter(tuple(ws[i:i + n]) for i in range(len(ws) - n + 1))
```

- `re.findall(r"[a-z']+", text.lower())` — lowercase runs of letters and apostrophes: punctuation and line breaks drop out.
- `tuple(ws[i:i + n])` — a tuple, because lists cannot be dictionary keys.
- `Counter(...)` — how many times each n-gram occurs.

bleu: clipped precisions, geometric mean, brevity penalty

MATH

$$p_n = \Sigma\, min(count_{hyp}(g), count_{ref}(g)) \,/\, \Sigma\, count_{hyp}(g)$$

$$BLEU = BP \cdot exp(\,(1/4)\, \Sigma_{n=1..4}\, log\, p_n\,), \quad BP = min(1, e^{1\,-\,r/c})$$

CODE

```
def bleu(hyps, refs, max_n=4):
    match, total = np.zeros(max_n), np.zeros(max_n)
    hyp_len = ref_len = 0
    for h, r in zip(hyps, refs):
        hw, rw = words(h), words(r); hyp_len += len(hw); ref_len += len(rw)
        for n in range(1, max_n + 1):
            hc, rc = ngrams(hw, n), ngrams(rw, n)
            match[n - 1] += sum(min(c, rc[g]) for g, c in hc.items())
            total[n - 1] += max(0, len(hw) - n + 1)
    if match[0] == 0:
        return 0.0, match / np.maximum(total, 1)
    prec = np.r_[match[0] / total[0], (match[1:] + 1) / (total[1:] + 1)]
    bp = 1.0 if hyp_len > ref_len else np.exp(1 - ref_len / max(hyp_len, 1))
    return bp * np.exp(np.mean(np.log(prec))), match / np.maximum(total, 1)
```

- `min(c, rc[g])` — clipping: an n-gram repeated five times counts at most as often as it appears in the reference.
- `match[0] == 0` — no shared word at all: BLEU is defined as 0 here (smoothing never applies to single words).
- `(match[1:] + 1) / (total[1:] + 1)` — BLEU+1: one added to the 2- to 4-gram counts, so a corpus with no shared 4-gram does not score exactly 0.
- `np.exp(1 - ref_len / max(hyp_len, 1))` — the brevity penalty, below 1 only when the hypotheses are shorter than the references.

rouge_n and rouge_l: recall and precision, combined as F1

MATH

$$R = overlap \,/\, |ref\ n\text{-}grams|, \quad P = overlap \,/\, |hyp\ n\text{-}grams|, \quad F1 = 2RP \,/\, (R + P)$$

$$LCS[i, j] = LCS[i{-}1, j{-}1] + 1 \text{ if } a_i = b_j, \text{ else } max(LCS[i{-}1, j], LCS[i, j{-}1])$$

CODE

```
def rouge_n(hyp, ref, n):
    hc, rc = ngrams(words(hyp), n), ngrams(words(ref), n)
    overlap = sum(min(c, rc[g]) for g, c in hc.items())
    rec = overlap / max(1, sum(rc.values())); prec = overlap / max(1, sum(hc.values()))
    return 0.0 if overlap == 0 else 2 * rec * prec / (rec + prec)
def rouge_l(hyp, ref):
    a, b = words(hyp), words(ref)
    L = np.zeros((len(a) + 1, len(b) + 1), dtype=int)
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            L[i, j] = L[i - 1, j - 1] + 1 if a[i - 1] == b[j - 1] else max(L[i - 1, j], L[i, j -
                1])
    lcs = L[-1, -1]
    return 0.0 if lcs == 0 else 2 * (lcs / len(b)) * (lcs / len(a)) / ((lcs / len(b)) + (lcs /
        len(a)))
```

- `rec = overlap / max(1, sum(rc.values()))` — recall: the share of the reference's n-grams the hypothesis has.

- `L[i, j] = L[i - 1, j - 1] + 1 if a[i - 1] == b[j - 1] else max(...)` — the dynamic-programming recurrence for the longest common subsequence.
- `L[-1, -1]` — the bottom-right cell: the LCS of the two whole word lists.

rouge: the three scores, averaged over pairs

CODE

```
def rouge(hyps, refs):
    return tuple(np.mean([f(h, r) for h, r in zip(hyps, refs)]) for f in (lambda h, r: rouge_n(h,
        r, 1),
                                                                          lambda h, r: rouge_n(h,
                                                                              r, 2), rouge_l))
```

- `lambda h, r: rouge_n(h, r, 1)` — a small unnamed function fixing n = 1, so all three scorers take (h, r).
- `tuple(np.mean(...) for f in (...))` — one mean per scorer, returned as (ROUGE-1, ROUGE-2, ROUGE-L).

Segment 3 — Models as functions

Loading a model through Day 96's part

CODE

```
def load_model(name):
    P = load_params(os.path.join(HERE, MODELS[name]))
    vocab = [str(v) for v in P.pop("vocab")]
    cfg = {k: int(v) for k, v in P.pop("cfg").items()}
    return to64(P), vocab, cfg
def to64(P):
    return {k: (to64(v) if isinstance(v, dict) else v.astype(np.float64)) for k, v in P.items()}
```

- `P.pop("vocab")` — removes the vocabulary from the dictionary and returns it, so only weights are left.
- `{k: int(v) for k, v in P.pop("cfg").items()}` — the settings were saved as 0-dimensional arrays; int() turns each into a number.
- `to64(v) if isinstance(v, dict) else ...` — recursion through the nesting: float32 on disk, float64 for the arithmetic.

gpt_logp, gpt_continue and kn_continue

CODE

```
def gpt_logp(P, cfg):
    return lambda x: d90.log_softmax(d90.gpt_forward(x, P, cfg["h"])[0])
def gpt_continue(P, cfg, prompts, n, rng, **how):
    seqs = [list(p) for p in prompts]
    for _ in range(n):
        lg = d90.gpt_forward(np.array([s[-cfg["T"]:] for s in seqs]), P, cfg["h"])[0][:, -1]
        for s, t in zip(seqs, sample_next(lg, rng, **how)):
            s.append(int(t))
    return [s[len(p):] for s, p in zip(seqs, prompts)]
def kn_continue(kn, vocab, prompt, n, rng, **how):
    text = prompt
    for _ in range(n):
        p = np.array([kn.prob(text[-(kn.n - 1):], c) for c in vocab])
        text += vocab[sample_next(np.log(p)[None], rng, **how)[0]]
    return text[len(prompt):]
```

- `lambda x: d90.log_softmax(d90.gpt_forward(x, P, cfg["h"])[0])` — a closure: the model packed into the one-argument function Day 92's eval_bits expects.

- `s[-cfg["T"]:]` — only the last T tokens fit in the model's context.
- `sample_next(np.log(p)[None], rng, **how)[0]` — Kneser-Ney's probabilities as log-scores in a batch of one, through the same sampler as the GPTs.

position_bits: the loss at each position of a window

MATH

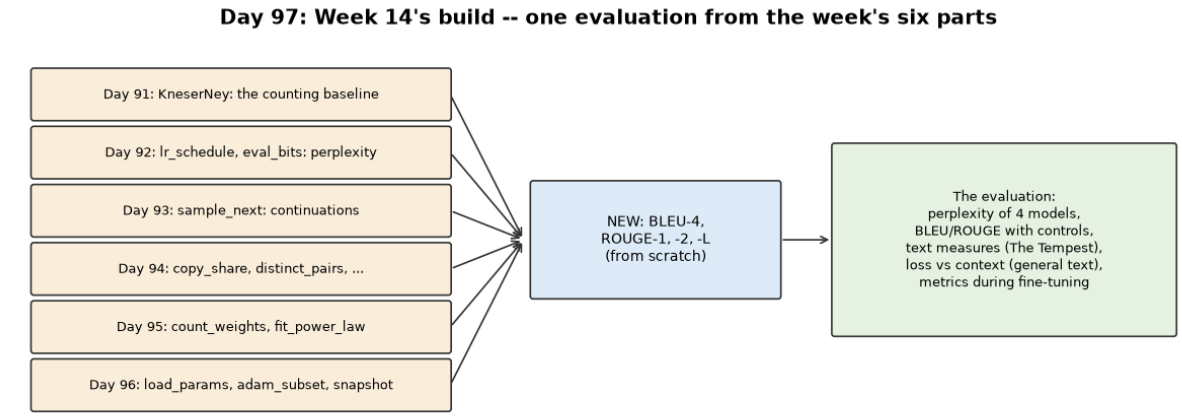
$$L(n) = \text{mean over windows of } -\log_2 p(x_{n+1} \mid x_1 \ldots x_n), \quad \text{fitted as } c + a\, n^{-\alpha}$$

CODE

```
def position_bits(logp_fn, ids_, T):
    n = (len(ids_) - 1) // T
    x, y = ids_[:n * T].reshape(n, T), ids_[1:n * T + 1].reshape(n, T)
    lp = np.concatenate([logp_fn(x[s:s + 32]) for s in range(0, n, 32)])
    return -np.take_along_axis(lp, y[..., None], axis=-1)[..., 0].mean(axis=0) / np.log(2)
```

- `ids_[:n * T].reshape(n, T)` — the text cut into n non-overlapping windows of T characters.
- `np.take_along_axis(lp, y[..., None], axis=-1)` — picks the log-probability of the true next character at every position.
- `.mean(axis=0)` — averages over windows, keeping one number per position: position p has seen p + 1 characters.

SYSTEM DIAGRAM



The six parts and the new metrics, assembled into one evaluation.

Segment 4 — The evaluation

demo_perplexity: four models, one test text

CODE

```
for name, text in (("Kneser-Ney, training text", tr_text), ("Kneser-Ney, + the play's train part",
    tr_text + ft_tr)):
    kn = KneserNey(text, KN_ORDER, KN_D); kns.append(kn); rows.append((name, kn.bits(ft_te,
        KN_ORDER - 1)))
for name, (P, vocab, cfg) in models.items():
    ix = {c: i for i, c in enumerate(vocab)}
    rows.append((name, eval_bits(gpt_logp(P, cfg), np.array([ix[c] for c in ft_te]),
        cfg["T"])[0]))
print(f"\n{'model':<36}{'bits/char':>10}{'ppl/char':>10}{'ppl/word':>10}")
for name, b in rows:
    print(f"{name:<36}{b:>10.3f}{2 ** b:>10.2f}{2 ** (b * cpw):>10.0f}")
```

- `kn.bits(ft_te, KN_ORDER - 1)` — Day 91's measure: every character from the 7th, with its real six characters of context.
- `eval_bits(gpt_logp(P, cfg), ...)[0]` — Day 92's full-context measure; [0] keeps the bits and drops the scored positions.
- `2 ** (b * cpw)` — perplexity per word: bits per character times characters per word, as a power of 2.

demo_bleu: continuations, references and controls

CODE

```
starts = np.arange(N_PROMPTS) * (PROMPT_LEN + CONT_LEN)
prompts = [ft_te[s:s + PROMPT_LEN] for s in starts]
refs = [ft_te[s + PROMPT_LEN:s + PROMPT_LEN + CONT_LEN] for s in starts]
other = refs[1:] + refs[:1]
...
controls = {"(reference itself)": refs, "(real text, wrong place)": other, "(prompt repeated)":
    [(p * 3)[:CONT_LEN] for p in prompts]}
...
for name, runs in systems.items():
    sc = np.array([(bleu(h, refs)[0],) + rouge(h, refs) for h in runs]); out[name] =
        sc.mean(axis=0)
```

- `other = refs[1:] + refs[:1]` — the references shifted by one: real Shakespeare, but the continuation of a different prompt.
- `(p * 3)[:CONT_LEN]` — the prompt repeated and cut to the continuation's length: a hypothesis that only echoes its input.
- `sc.mean(axis=0), sc.std(axis=0)` — mean and spread over the sampling seeds, per metric.

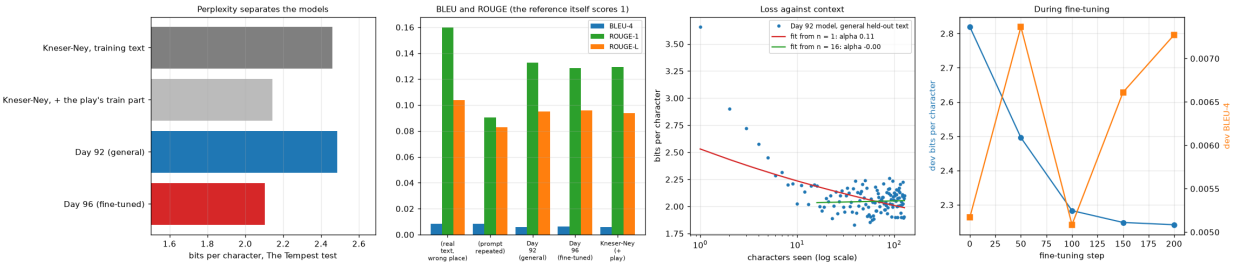
demo_training_signal: two metrics while fine-tuning

CODE

```
def check():
    hyps = ["".join(vocab[i] for i in c) for c in gpt_continue(P, cfg, prompts, CONT_LEN,
        np.random.RandomState(0), top_p=0.9)]
    return eval_bits(gpt_logp(P, cfg), dev, cfg["T"])[0], bleu(hyps, refs)[0]
...
rng = np.random.RandomState(0); state = {"t": 0, "m": {}, "v": {}}
T, B = cfg["T"], cfg["B"]
rows = [(0,) + check()]; best = (rows[0][1], 0, snapshot(P))
for step in range(1, FT_STEPS + 1):
    o = rng.randint(0, len(tr) - T - 1, B)
    x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
    _, G = d90.gpt_loss_grads(x, y, P, h=cfg["h"])
    adam_subset(P, G, state, lr_schedule(step - 1, FT_LR, 10, FT_STEPS), keys)
    if step % FT_CHECK == 0:
        rows.append((step,) + check())
        if rows[-1][1] < best[0]:
            best = (rows[-1][1], step, snapshot(P))
```

- `def check():` — a nested function: it sees P, prompts and refs from the enclosing function.
- `np.random.RandomState(0)` — the same sampling seed at every check, so BLEU changes only because the model does.
- `adam_subset(P, G, state, lr_schedule(...), keys)` — Day 96's and Day 92's parts together: only the top block and output layer move.

REAL GRAPH



Real graph from the script: perplexity, BLEU/ROUGE with controls, loss against context, and both metrics during fine-tuning.

Quick Syntax Glossary — new or reinforced constructs

- `Counter(iterable)` — a dictionary of counts; a missing key counts 0.
- `tuple(list)` — an immutable sequence, usable as a dictionary key.
- `d.pop(key)` — remove a key from a dictionary and return its value.
- `lambda x: ...` — a small unnamed function, here a model wrapped as ids -> log-probabilities.
- `np.take_along_axis(a, idx, axis)` — pick one entry per row along an axis, by index.
- `ax.twinx()` — a second y-axis on the same panel, for two quantities with different scales.

End of Day 97 syntax reference. For the experiments and what they showed, see the Day 97 complete guide.